LEKCJA 1

Lekcja 1: Wprowadzenie do Data Science i Machine Learning

#lekcja #datascience #wprowadzenie #machine-learning #ai #podstawy

LearnIT

learnit.com.pl

Spis treści

1 Czym jest Data Science, Machine Learning i AI?
2 Typy zadań w Machine Learning
3 Podejścia do uczenia modeli ML
4 🌱 Aspekt środowiskowy
5 🧪 Zadania do samodzielnej pracy
6 📚 Podsumowanie
7 📖 Dodatkowe Zasoby

Witamy na pierwszej lekcji kursu Data Science! W tej lekcji poznasz podstawowe pojęcia związane z nauką o danych, uczeniem maszynowym i sztuczną inteligencją. Dowiesz się, czym różnią się te dziedziny, jakie problemy możemy rozwiązywać za pomocą Machine Learning oraz jak dane mogą wspierać lepsze decyzje biznesowe i proekologiczne. To fundament, na którym zbudujesz swoją karierę w Data Science!

---

1. Czym jest Data Science, Machine Learning i AI?

> 📚 **Definicja**
>
> **Data Science (Nauka o Danych)** to interdyscyplinarna dziedzina, która łączy metody statystyczne, programowanie i wiedzę domenową w celu wydobywania wiedzy i wartości z danych.

Wprowadzenie do ekosystemu AI/ML/DS

W dzisiejszym świecie technologii często słyszymy pojęcia takie jak **Sztuczna Inteligencja (AI)**, **Machine Learning (ML)** i **Data Science (DS)**. Choć są ze sobą ściśle powiązane, każde z nich ma swoją specyfikę:

Sztuczna Inteligencja (AI - Artificial Intelligence)

- Najszersze pojęcie
- Obejmuje wszelkie systemy, które potrafią wykonywać zadania wymagające "ludzkiej inteligencji"
- Przykłady: rozpoznawanie mowy, rozumienie języka naturalnego, gra w szachy, autonomiczne pojazdy

Machine Learning (ML - Uczenie Maszynowe)

- Podzbiór AI
- Systemy, które **uczą się z danych** bez bezpośredniego programowania każdej reguły
- Model "uczy się" wzorców z przykładów i potrafi uogólniać na nowe przypadki
- Przykłady: spam filter, rekomendacje filmów Netflix, rozpoznawanie twarzy

Data Science (DS - Nauka o Danych)

- Interdyscyplinarna dziedzina
- Łączy statystykę, programowanie, ML i wiedzę biznesową
- Cały proces: od zbierania danych → czyszczenia → analizy → modelowania → wizualizacji → wdrożenia
- Przykłady: analiza zachowań klientów, prognozowanie sprzedaży, optymalizacja procesów

Diagram relacji AI, ML i DS

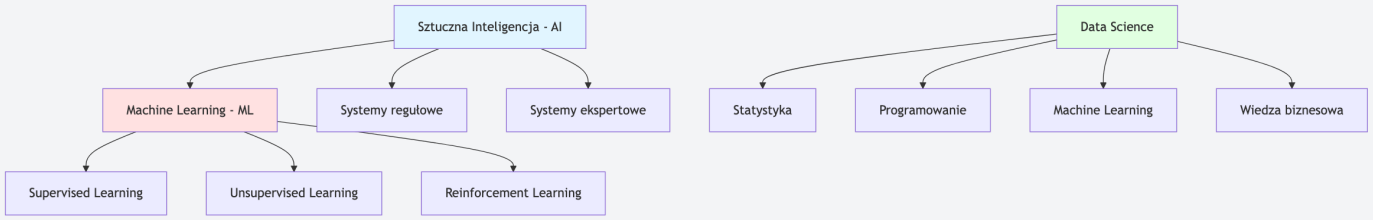


Diagram 1

Przykład 1: Porównanie podejść do tego samego problemu

Wyobraź sobie, że chcemy stworzyć system do **klasyfikacji emaili jako spam lub nie-spam**.

Podejście 1: Tradycyjne programowanie (reguły)

```
# Tradycyjne podejście - ręcznie tworzone reguły
def is_spam_traditional(email_text):
    spam_words = ['viagra', 'casino', 'win money', 'click here']

    # Sprawdzamy obecność słów kluczowych
    for word in spam_words:
        if word in email_text.lower():
            return True  # To spam
    return False  # To nie spam

# Przykład użycia
email = "Click here to win money at our casino!"
print(f"Czy to spam? {is_spam_traditional(email)}")
# Wynik: Czy to spam? True
```

> ⚠️ **Uwaga**
>
> **Problem z regułami:** Spamerzy szybko nauczą się omijać te konkretne słowa (np. "v1agra", "cas1no"). Musielibyśmy ciągle aktualizować reguły ręcznie!

Podejście 2: Machine Learning

```
# Podejście ML - model uczy się wzorców z przykładów (pseudo-kod)
# 1. Zbieramy tysiące przykładów emaili oznaczonych jako spam/nie-spam
training_data = [
    ("Spotkanie o 15:00", "nie-spam"),
    ("Click here to win!", "spam"),
    ("Raport miesięczny w załączniku", "nie-spam"),
    ("Free viagra!!!", "spam"),
    # ... tysiące innych przykładów
]

# 2. Model ML "uczy się" wzorców
model = train_ml_model(training_data)

# 3. Model potrafi klasyfikować NOWE emaile, których nigdy nie widział
new_email = "Cl1ck h3re to w1n m0ney"
prediction = model.predict(new_email)
# Model rozpozna spam mimo zmienionych liter!
```

> ✅ **Sukces**
>
> **Zaleta ML:** Model sam wykrywa wzorce i potrafi generalizować na nowe, nigdy nie widziane przypadki. Nie musimy ręcznie tworzyć każdej reguły!

Przykład 2: Data Science w praktyce - pełny cykl

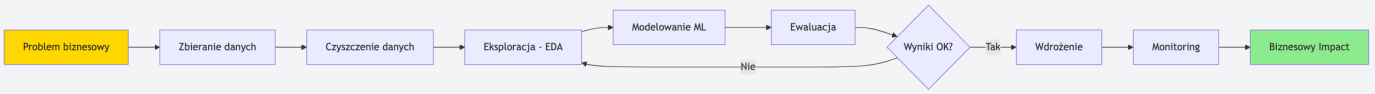


Diagram 2

Przykład projektu DS: Problem biznesowy: Sklep internetowy traci klientów - chcemy zrozumieć dlaczego i jak to zmienić. **Proces Data Science:**

1. **Zbieranie danych:**

- Historia zamówień klientów
- Dane o wizytach na stronie
- Dane demograficzne
- Recenzje produktów

1. **Czyszczenie danych:**

- Usunięcie duplikatów
- Obsługa brakujących wartości
- Korekta błędnych danych (np. negatywna cena)

1. **Eksploracja (EDA - Exploratory Data Analysis):**

- Wizualizacje: wykresy sprzedaży w czasie
- Statystyki: średnia wartość koszyka
- Odkrycia: klienci rezygnują po 3 miesiącach

1. **Modelowanie ML:**

- Model przewidujący prawdopodobieństwo rezygnacji klienta
- Identyfikacja czynników ryzyka

1. **Wdrożenie:**

- System alertów dla działu retention
- Automatyczne kampanie marketingowe dla zagrożonych klientów

1. **Monitoring:**

- Czy model działa dobrze w produkcji?
- Czy spadła liczba rezygnacji?

> 📝 **Notatka**
>
> Data Scientist musi rozumieć biznes, dane, statystykę i programowanie. To profesja interdyscyplinarna!

Przykład 3: Realne zastosowania AI/ML/DS

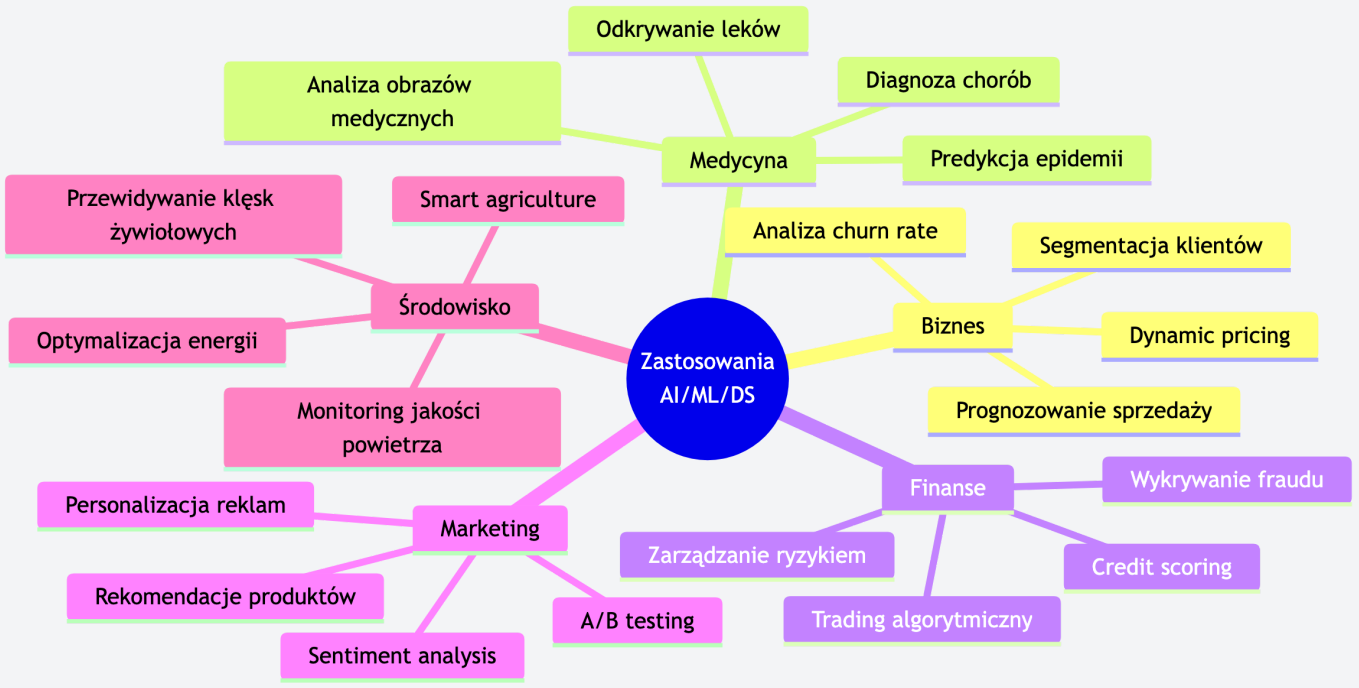


Diagram 3

Przykłady z różnych branż: E-commerce (Netflix, Amazon):

- Systemy rekomendacji: "Klienci, którzy kupili X, kupili też Y"
- Personalizacja doświadczeń użytkownika
- Optymalizacja cen i promocji

Finanse (banki, fintechy):

- Wykrywanie transakcji fraudulentnych w czasie rzeczywistym
- Ocena zdolności kredytowej (credit scoring)
- Algorytmiczna wymiana walut

Medycyna:

- Analiza zdjęć RTG/CT w poszukiwaniu nowotworów
- Przewidywanie skuteczności leczenia
- Odkrywanie nowych leków metodami AI

Transport (Uber, autonomiczne pojazdy):

- Prognozowanie czasu przejazdu
- Optymalizacja tras
- Systemy autonomicznej jazdy

Ekologia i środowisko:
- Monitoring zanieczyszczeń powietrza
- Optymalizacja zużycia energii w budynkach
- Prognozowanie zmian klimatycznych

> 💡 **Wskazówka**
>
> Data Science to nie tylko technologia - to narzędzie do rozwiązywania realnych problemów ludzi i biznesów!

---

2. Typy zadań w Machine Learning

> 📚 **Definicja**
>
> **Zadanie ML** to typ problemu, który chcemy rozwiązać za pomocą uczenia maszynowego. Różne zadania wymagają różnych podejść i algorytmów.

Wprowadzenie

Machine Learning może rozwiązywać wiele różnych typów problemów. Kluczowe jest rozpoznanie, z jakim typem zadania mamy do czynienia, ponieważ to determinuje wybór algorytmu i metodologii.

Główne typy zadań ML

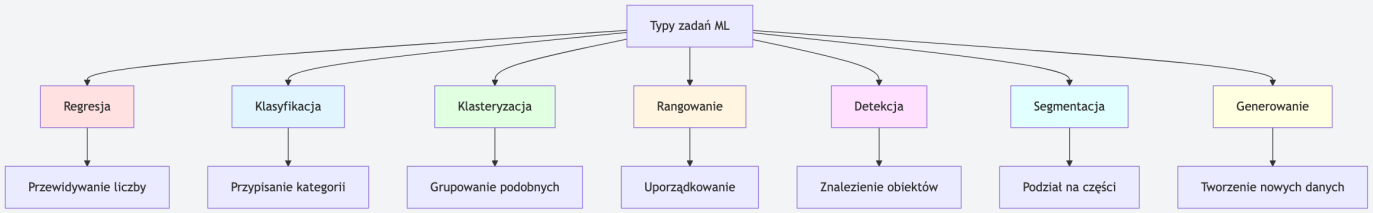


Diagram 4

1. Regresja - przewidywanie wartości numerycznej

Definicja: Przewidujemy **konkretną liczbę** (wartość ciągłą). **Przykłady:**

- Przewidywanie ceny mieszkania (np. 450,000 zł)
- Prognozowanie temperatury za tydzień (np. 18.5°C)
- Szacowanie czasu dostawy (np. 35 minut)
- Przewidywanie sprzedaży w następnym miesiącu (np. 15,230 produktów)

Ilustracja:



Diagram 5

Prosty przykład - przewidywanie ceny mieszkania:

```
# Dane wejściowe (cechy mieszkania)
powierzchnia = 65  # m²
```

```
liczba_pokoi = 3
odleglosc_od_centrum = 2  # km

# Model nauczył się wzorców z tysięcy przykładów i przewiduje
przewidywana_cena = model_regresji.predict(powierzchnia, liczba_pokoi,
odleglosc_od_centrum)
# Wynik: 480,000 zł
```

> 📝 Notatka
>
> W regresji wynik to zawsze **liczba**, nie kategoria!

2. Klasyfikacja - przypisanie do kategorii

Definicja: Przypisujemy obiekt do jednej z **predefiniowanych kategorii/klas**. **Klasyfikacja binarna (2 klasy):**

- Email: spam / nie-spam
- Transakcja: fraudulentna / legalna
- Pacjent: chory / zdrowy
- Film: polecany / niepolecany

Klasyfikacja wieloklasowa (>2 klasy):

- Rozpoznawanie cyfr 0-9 na zdjęciu
- Kategoryzacja artykułów: sport / polityka / technologia / rozrywka
- Klasyfikacja gatunku zwierzęcia: pies / kot / ptak / ryba
- Ocena jakości produktu: niska / średnia / wysoka / premium

Ilustracja:

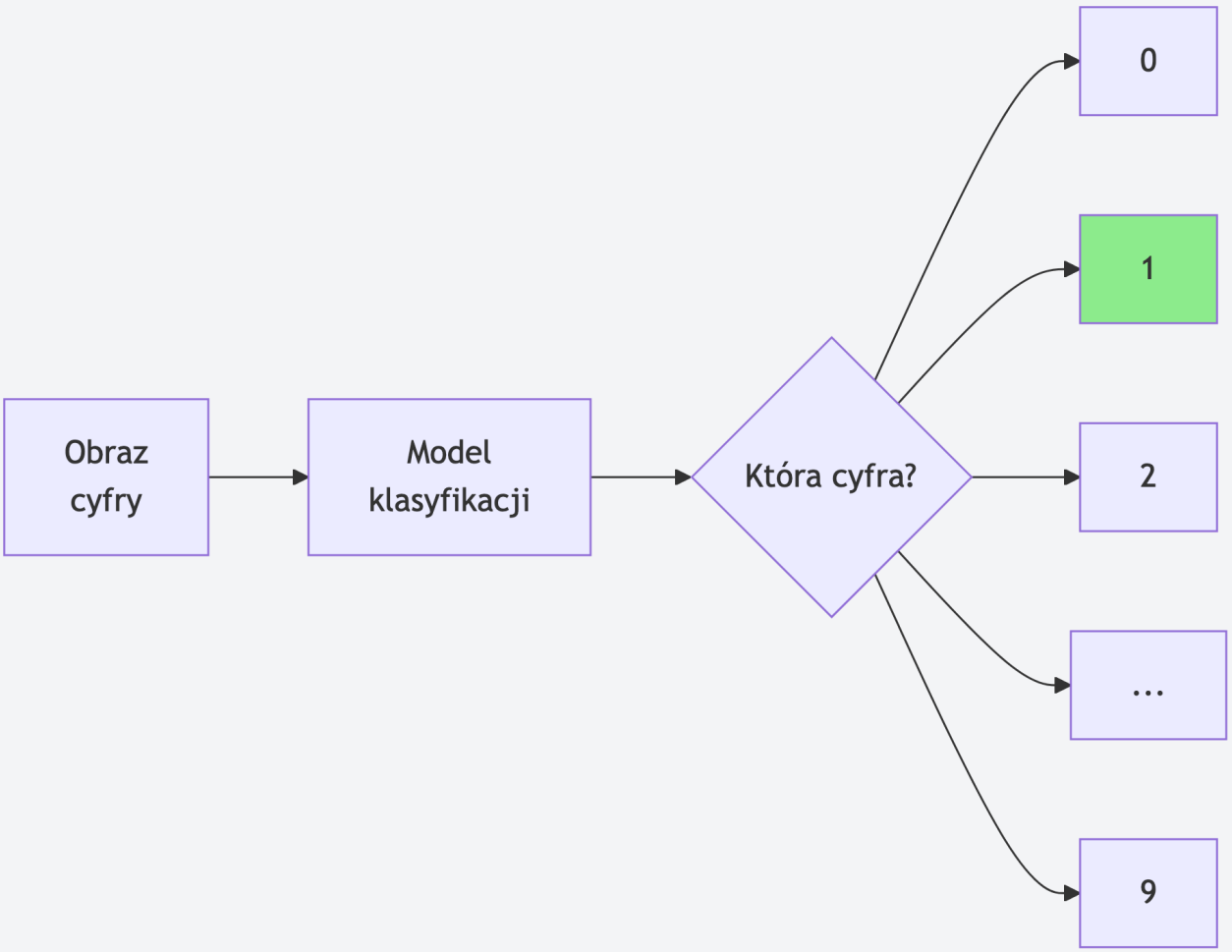


Diagram 6

Przykład - klasyfikacja emaili:

```
# Dane wejściowe
email_text = "Congratulations! You won $1,000,000! Click here now!!!"

# Model analizuje cechy tekstu i klasyfikuje
prediction = model_klasyfikacji.predict(email_text)
probability_spam = 0.95  # 95% pewności, że to spam

print(f"Klasyfikacja: {prediction}")  # Wynik: "SPAM"
print(f"Pewność: {probability_spam}")  # Wynik: 0.95
```

> 💡 **Wskazówka**
>
> Klasyfikacja często zwraca też **prawdopodobieństwo** dla każdej klasy, nie tylko ostateczną decyzję!

3. Klasteryzacja - grupowanie podobnych obiektów

Definicja: Dzielimy dane na **grupy (clustery)** obiektów podobnych do siebie, bez wcześniejszego określenia kategorii. **Przykłady:**

- Segmentacja klientów na grupy według zachowań zakupowych
- Grupowanie artykułów tematycznych
- Identyfikacja typów użytkowników aplikacji
- Grupowanie genów o podobnej ekspresji

Różnica vs klasyfikacja:

- **Klasyfikacja:** Znamy kategorie z góry (spam/nie-spam)
- **Klasteryzacja:** Model **sam odkrywa** naturalne grupy w danych

Ilustracja - segmentacja klientów:

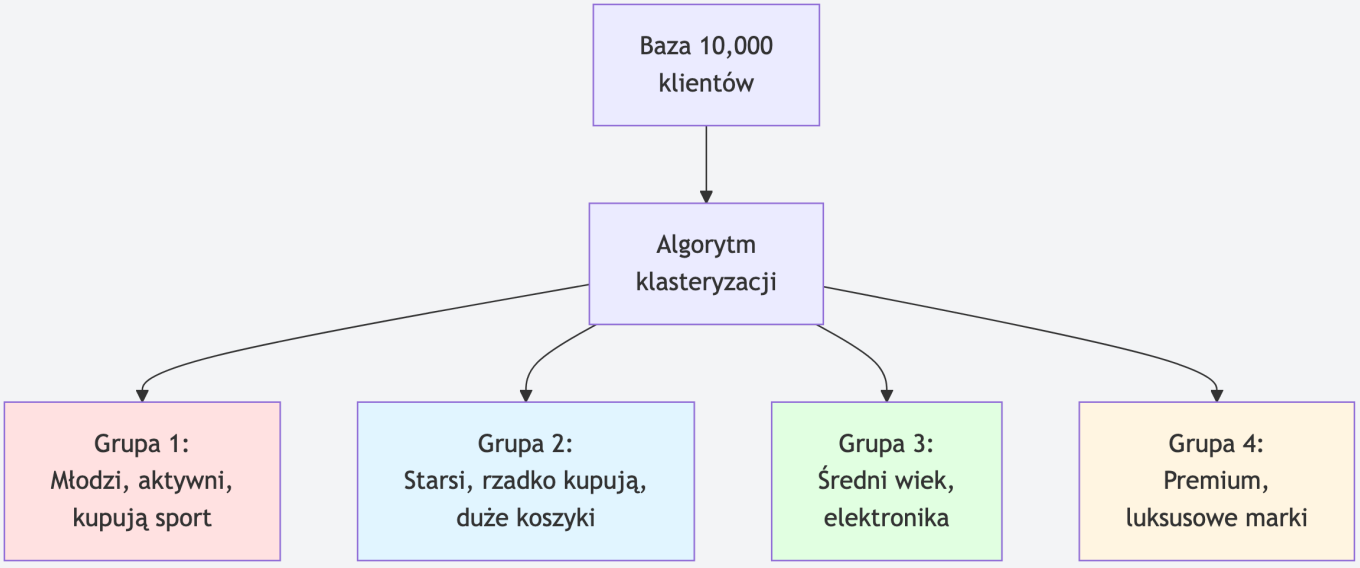


Diagram 7

Przykład - segmentacja klientów sklepu:

```
# Dane o klientach: wiek, dochód, częstotliwość zakupów, średnia wartość koszyka
klienci_data = [
    [25, 3000, 12, 150],   # Klient 1
    [45, 8000, 5, 800],    # Klient 2
    [30, 3500, 15, 120],   # Klient 3
    # ... tysiące klientów
]

# Model klasteryzacji znajduje naturalne grupy
clusters = model_klasteryzacji.fit(klienci_data)

# Wynik: 4 grupy klientów o podobnych cechach
# Grupa 0: Młodzi, niski dochód, często kupują małe kwoty
# Grupa 1: Starsi, wysoki dochód, rzadko ale duże zakupy
# Grupa 2: Średni wiek, średni dochód, regularni
# Grupa 3: Premium - VIP klienci
```

> ⚠ Uwaga
>
> Klasteryzacja to **uczenie nienadzorowane** - nie mamy gotowych etykiet, model sam musi znaleźć strukturę w danych!

4. Rangowanie - uporządkowanie według trafności

Definicja: Układamy obiekty w **kolejności od najbardziej do najmniej istotnego/trafnego** dla danego kryterium. **Przykłady:**

- Wyniki wyszukiwania Google (najlepsze wyniki na górze)
- Rekomendacje produktów w e-commerce
- Rankingi filmów/restauracji
- Ranking kandydatów do pracy

Ilustracja:

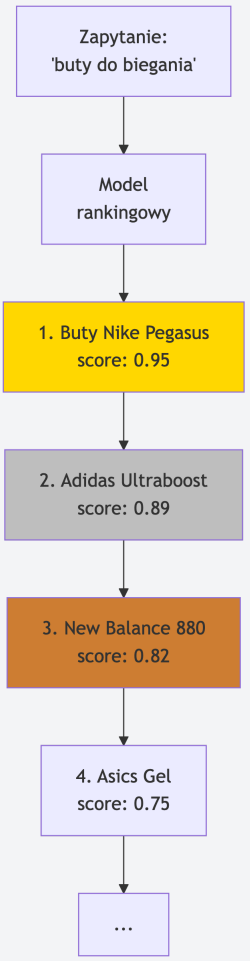


Diagram 8

Przykład - system rekomendacji:

```
# Użytkownik szuka: "laptop do programowania"
query = "laptop do programowania"

# Model ranguje produkty według trafności
produkty = [
    {"nazwa": "MacBook Pro M3", "relevance_score": 0.95},
    {"nazwa": "Dell XPS 15", "relevance_score": 0.92},
    {"nazwa": "ThinkPad X1", "relevance_score": 0.88},
    {"nazwa": "Tablet iPad", "relevance_score": 0.35},  # Mało trafny
]

# Sortujemy według score
```

```
sorted_produkty = sort_by_score(produkty)
# Użytkownik widzi najpierw najbardziej trafne wyniki
```

> ✅ **Sukces**
>
> Dobre rangowanie to podstawa wyszukiwarek i systemów rekomendacji - kluczowe dla doświadczenia użytkownika!

5. Detekcja - wykrywanie obiektów

Definicja: Znajdowanie i **lokalizowanie obiektów** w danych (najczęściej obrazach lub wideo). **Przykłady:**

- Wykrywanie twarzy na zdjęciu
- Rozpoznawanie numerów rejestracyjnych (OCR)
- Detekcja samochodów/pieszych w autonomicznych pojazdach
- Wykrywanie nowotworów na zdjęciach medycznych

Ilustracja:

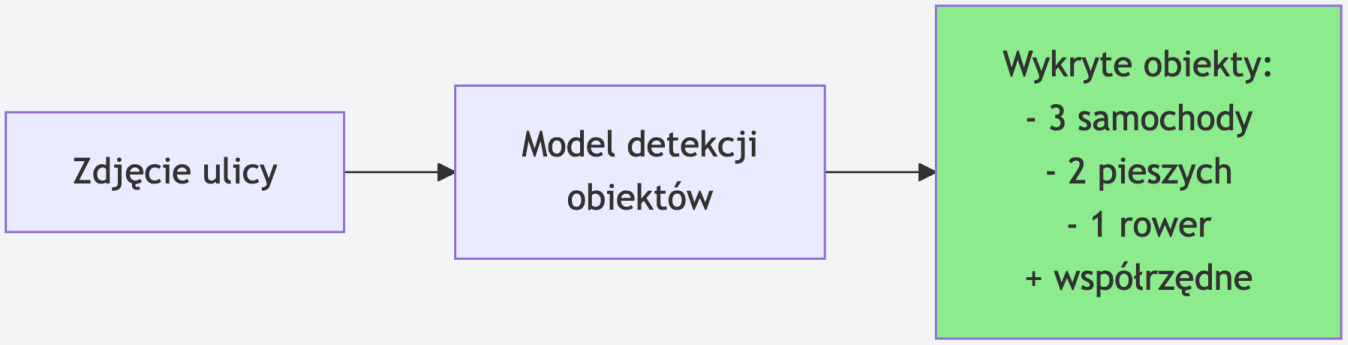


Diagram 9

Przykład konceptualny - detekcja obiektów:

```
# Obraz z kamery samochodu autonomicznego
image = load_image("ulica.jpg")
```

```
# Model detekcji obiektów
detections = model_detekcji.detect(image)

# Wynik:
# [
#   {"obiekt": "samochód", "pozycja": (120, 200, 300, 400), "pewność": 0.98},
#   {"obiekt": "pieszy", "pozycja": (450, 300, 520, 600), "pewność": 0.92},
#   {"obiekt": "rower", "pozycja": (600, 250, 680, 450), "pewność": 0.85}
# ]

# Samochód wie, gdzie są obiekty i może bezpiecznie jechać
```

> 📝 Notatka
>
> Detekcja nie tylko rozpoznaje **co** jest na obrazie, ale też **gdzie** się to znajduje!

6. Segmentacja - podział obrazu na części

Definicja: Dzielimy obraz na **regiony/piksele** należące do różnych obiektów lub kategorii. **Przykłady:**

- Medycyna: wydzielenie tkanki nowotworowej na skanach MRI
- Autonomiczne pojazdy: rozróżnienie drogi, chodnika, nieba, budynków
- Edycja zdjęć: automatyczne zaznaczanie tła do usunięcia
- Analiza satelitarna: identyfikacja lasów, pól, miast

Różnica vs detekcja:

- **Detekcja:** Prostokąt wokół obiektu ("tutaj jest pies")
- **Segmentacja:** Dokładny kontur obiektu ("te konkretne piksele to pies")

Ilustracja:

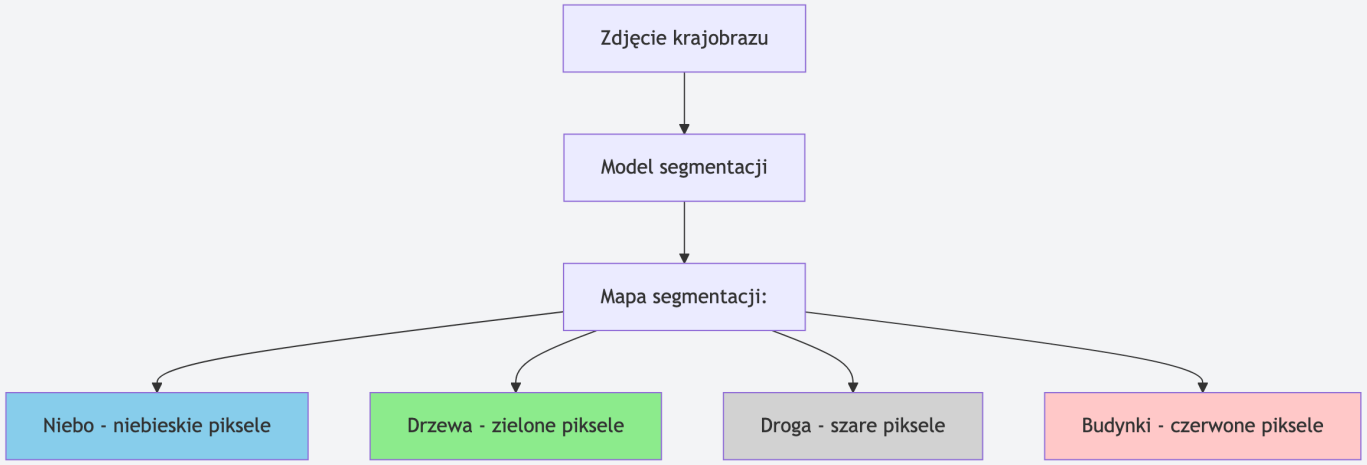


Diagram 10

Przykład konceptualny:

```
# Zdjęcie medyczne - MRI mózgu
mri_scan = load_image("brain_mri.jpg")

# Model segmentacji wykrywa różne tkanki
segmentation_mask = model_segmentacji.segment(mri_scan)

# Wynik: maska, gdzie każdy piksel ma przypisaną klasę
# Klasa 0: tło
# Klasa 1: zdrowa tkanka
# Klasa 2: tkanka podejrzana (możliwy nowotwór)
# Klasa 3: płyn mózgowo-rdzeniowy

# Lekarz widzi dokładnie, które obszary są podejrzane
```

> 💡 **Wskazówka**
>
> Segmentacja jest bardziej precyzyjna niż detekcja - kluczowa w medycynie i zastosowaniach wymagających dokładności!

7. Generowanie - tworzenie nowych danych

Definicja: Model **tworzy nowe dane** podobne do danych treningowych - teksty, obrazy, muzykę, kod.

Przykłady:

- Generowanie tekstu (ChatGPT, GPT-4)
- Tworzenie obrazów (DALL-E, Midjourney, Stable Diffusion)
- Generowanie muzyki
- Synteza głosu
- Tworzenie kodu programistycznego

Ilustracja:

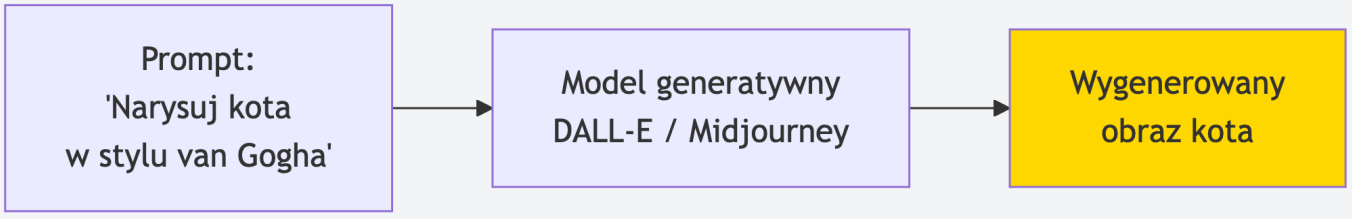


Diagram 11

Przykład - generowanie tekstu:

```
# Prompt dla modelu
prompt = "Napisz krótki wiersz o jesieni"

# Model generatywny tworzy tekst
generated_text = model_generatywny.generate(prompt)

# Wynik:
# "Złote liście spadają z drzew,
#  Wiatr szumi w koronach.
#  Jesień maluje świat w barwy ciepłe,
#  Przygotowując nas do zimy."

# Model stworzył NOWY tekst, którego nigdy wcześniej nie było!
```

⚠️ Uwaga

Modele generatywne to najbardziej zaawansowane systemy AI - wymagają ogromnych zasobów obliczeniowych i danych treningowych!

Podsumowanie typów zadań

Typ zadania	Pytanie	Przykład wyniku
Regresja	Ile? Jaka wartość?	450,000 zł
Klasyfikacja	Do jakiej kategorii?	"Spam"
Klasteryzacja	Jakie są naturalne grupy?	4 segmenty klientów
Rangowanie	W jakiej kolejności?	Lista od najbardziej trafnych
Detekcja	Gdzie są obiekty?	Współrzędne: (120, 200, 300, 400)
Segmentacja	Które piksele do czego należą?	Maska segmentacji
Generowanie	Stwórz coś nowego	Wygenerowany obraz/tekst

✅ Sukces

Rozpoznanie typu zadania to pierwszy krok w każdym projekcie ML - determinuje wybór algorytmu i metodologii!

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

3. Podejścia do uczenia modeli ML

📚 Definicja

Podejście do uczenia określa, **w jaki sposób** model uczy się z danych - czy potrzebuje etykiet, czy uczy się metodą prób i błędów, czy też kombinuje różne strategie.

Wprowadzenie

Nie wszystkie problemy ML możemy rozwiązać w ten sam sposób. W zależności od tego, jakie dane mamy dostępne i czego chcemy się nauczyć, wybieramy odpowiednie podejście do trenowania modelu.

Główne typy uczenia

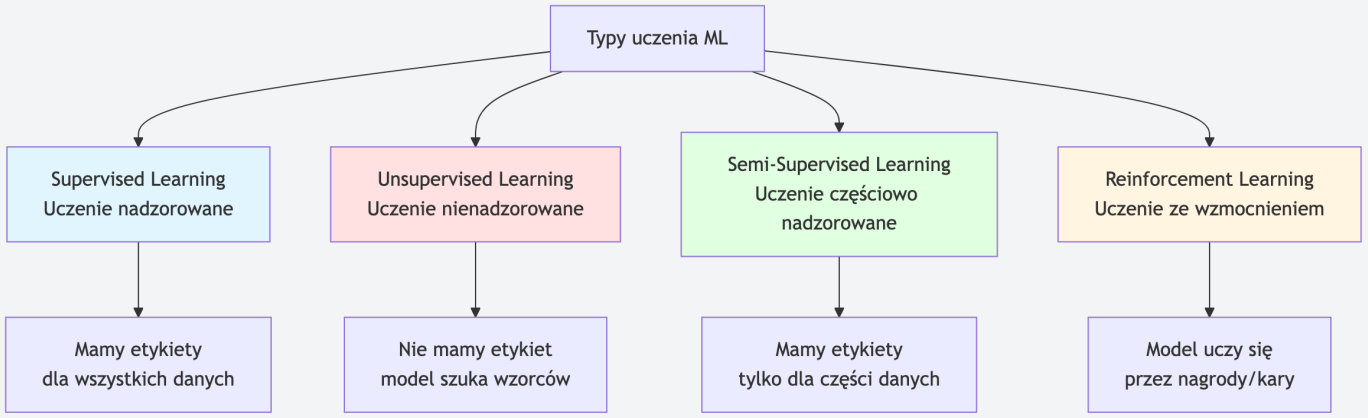


Diagram 12

1. Supervised Learning - Uczenie nadzorowane

> 📚 **Definicja**
>
> W **uczeniu nadzorowanym** model uczy się z **danych oznaczonych (labeled data)** - mamy zarówno dane wejściowe (X), jak i prawidłowe odpowiedzi (y).

Jak to działa:

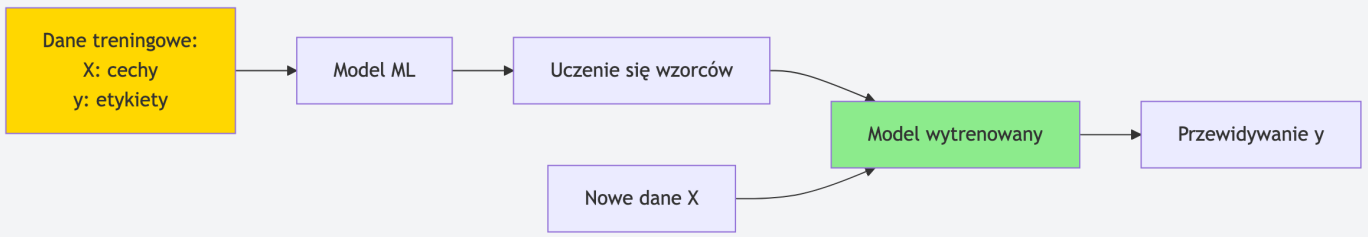


Diagram 13

Przykład - klasyfikacja emaili: Dane treningowe (oznaczone przez ludzi):

```
training_data = [
    # (tekst emaila, etykieta)
    ("Spotkanie o 15:00 w sali konferencyjnej", "NIE-SPAM"),
    ("Wygrałeś 1,000,000 PLN! Kliknij tutaj", "SPAM"),
    ("Raport miesięczny w załączniku", "NIE-SPAM"),
    ("Free Viagra!!! Click now!!!", "SPAM"),
    # ... tysiące przykładów z etykietami
]

# Model uczy się: jakie cechy tekstu wskazują na spam vs nie-spam
model = train_supervised(training_data)

# Teraz model może klasyfikować NOWE emaile
new_email = "Limited time offer! Buy now!"
prediction = model.predict(new_email)  # Wynik: "SPAM"
```

Zastosowania Supervised Learning:

- Klasyfikacja (spam detection, rozpoznawanie obrazów)
- Regresja (przewidywanie cen, prognozowanie sprzedaży)
- Ranking (systemy rekomendacji)

Zalety:

- ✅ Wysoka dokładność (jeśli mamy dobre etykiety)
- ✅ Jasna metoda ewaluacji (porównujemy przewidywania z prawdą)

Wady:

- ❌ Wymaga **dużo oznaczonych danych** (czasochłonne i kosztowne)
- ❌ Etykietowanie przez ludzi może być drogie

> 📝 Notatka
>
> Większość praktycznych zastosowań ML to supervised learning!

2. Unsupervised Learning - Uczenie nienadzorowane

> **Definicja**
>
> W **uczeniu nienadzorowanym** model uczy się z **danych nieoznaczonych** - mamy tylko dane wejściowe (X), bez etykiet. Model sam musi odkryć strukturę i wzorce w danych.

Jak to działa:

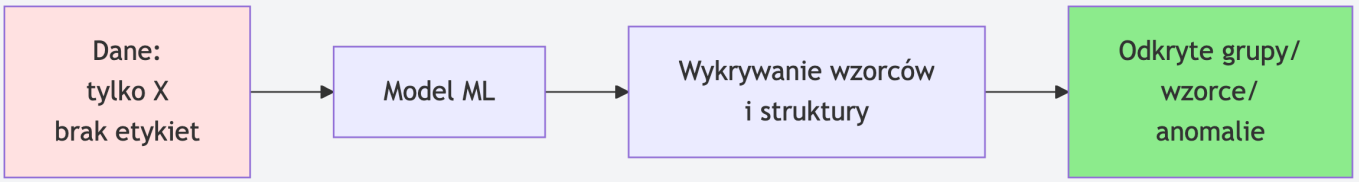


Diagram 14

Przykład - klasteryzacja klientów:

```
# Dane o klientach - BEZ ETYKIET!
# Nie wiemy z góry, jakie grupy istnieją
customers_data = [
    [25, 3000, 12],   # wiek, dochód, częstotliwość zakupów
    [45, 8000, 5],
    [30, 3500, 15],
    [50, 9000, 4],
    [28, 3200, 10],
    # ... tysiące klientów
]

# Model klasteryzacji SAM znajduje naturalne grupy
clusters = unsupervised_clustering(customers_data)

# Wynik: model znalazł 3 grupy:
# Cluster 0: Młodzi, niski dochód, często kupują
# Cluster 1: Starsi, wysoki dochód, rzadko kupują
# Cluster 2: Średni wiek, średni dochód, regularni
```

```
# Nikt nie mówił modelowi, ile grup istnieje - SAM TO ODKRYŁ!
```

Zastosowania Unsupervised Learning:

- Klasteryzacja (segmentacja klientów, grupowanie dokumentów)
- Redukcja wymiarowości (kompresja danych, wizualizacja)
- Wykrywanie anomalii (fraud detection, błędy w produkcji)
- Association rules (analiza koszyka zakupowego: "klienci kupujący X często kupują Y")

Zalety:

- ✅ Nie wymaga kosztownego etykietowania
- ✅ Odkrywa nieoczekiwane wzorce
- ✅ Działa z dużymi zbiorami nieoznaczonych danych

Wady:

- ❌ Trudniejsza ewaluacja (brak "prawdy" do porównania)
- ❌ Wyniki mogą być trudne do interpretacji

> 💡 **Wskazówka**
>
> Unsupervised learning świetnie nadaje się do eksploracji danych i odkrywania nowych insights!

3. Semi-Supervised Learning - Uczenie częściowo nadzorowane

> 📚 **Definicja**
>
> **Uczenie częściowo nadzorowane** łączy oba podejścia - wykorzystuje **małą ilość danych oznaczonych** i **dużą ilość danych nieoznaczonych**.

Kiedy używamy:

- Mamy tysiące/miliony przykładów, ale oznaczenie wszystkich jest drogie
- Przykład: mamy 1,000,000 zdjęć, ale tylko 10,000 jest opisanych

Jak to działa:

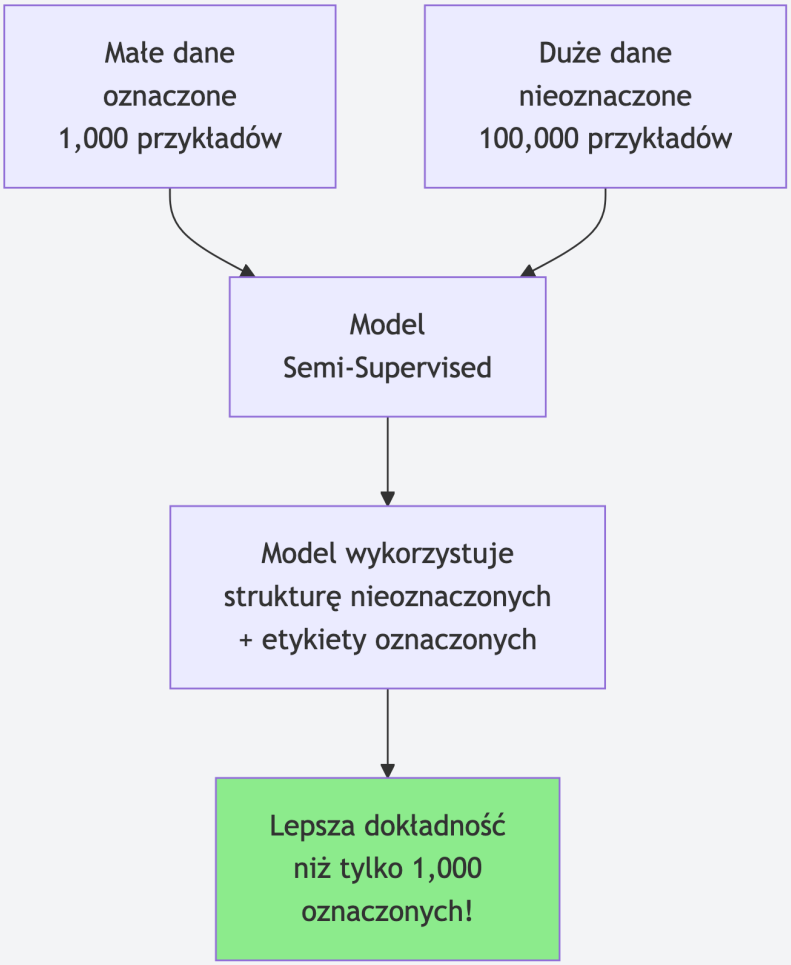


Diagram 15

Przykład - rozpoznawanie obiektów na zdjęciach:

```
# Mamy:
# - 1,000 zdjęć z etykietami (opisane przez ludzi)
# - 100,000 zdjęć BEZ etykiet

labeled_images = load_labeled_data(1000)    # Kosztowne do przygotowania
unlabeled_images = load_unlabeled_data(100000)  # Łatwe do zdobycia

# Model uczy się:
# 1. Z oznaczonych zdjęć - jakie obiekty występują
# 2. Z nieoznaczonych - jakie wzorce i struktury są w danych

model = train_semi_supervised(labeled_images, unlabeled_images)
```

```
# Wynik: model działa lepiej niż gdyby uczył się TYLKO z 1,000 oznaczonych!
```

Zalety:

- ✅ Wysoka dokładność przy mniejszym koszcie etykietowania
- ✅ Wykorzystuje łatwo dostępne dane nieoznaczone

Wady:

- ❌ Bardziej złożone algorytmy
- ❌ Wciąż wymaga PEWNEJ ilości oznaczonych danych

> ✅ **Sukces**
>
> Semi-supervised learning to świetny kompromis - jakość supervised learning przy kosztach bliższych unsupervised!

4. Reinforcement Learning - Uczenie ze wzmocnieniem

> 📚 **Definicja**
>
> W **uczeniu ze wzmocnieniem** model (agent) uczy się przez **interakcję ze środowiskiem** - otrzymuje nagrody za dobre decyzje i kary za złe.

Jak to działa:

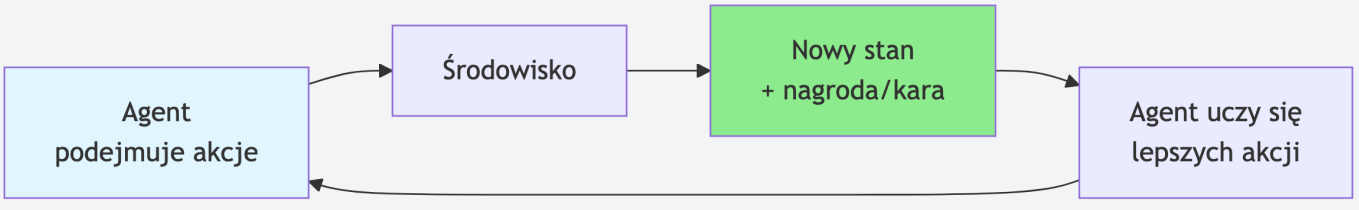


Diagram 16

Kluczowe elementy:

- **Agent:** Model ML, który podejmuje decyzje
- **Środowisko:** Świat, w którym agent działa
- **Stan:** Aktualna sytuacja
- **Akcja:** Co agent może zrobić
- **Nagroda:** Feedback za akcję (dobra/zła)

Przykład - gra w szachy:

```
# Agent: program szachowy
# Środowisko: plansza szachowa
# Stan: aktualne ułożenie figur
# Akcje: możliwe ruchy
# Nagroda: +1 za wygraną, -1 za przegraną, 0 za remis

for episode in range(1000000):  # Milion gier treningowych
    state = chess_game.reset()  # Nowa gra

    while not game_over:
        action = agent.choose_move(state)  # Agent wybiera ruch
        new_state, reward, done = chess_game.step(action)

        # Agent uczy się:
        # - Jeśli wygrał (reward=+1) -> te ruchy były dobre
        # - Jeśli przegrał (reward=-1) -> te ruchy były złe

        agent.learn(state, action, reward, new_state)
        state = new_state

# Po milionach gier agent staje się bardzo dobry!
```

Zastosowania Reinforcement Learning:

- Gry (AlphaGo, szachy, Dota 2)
- Roboty (chód, manipulacja obiektami)
- Autonomiczne pojazdy (jazda w ruchu miejskim)
- Optymalizacja procesów (zarządzanie energią, logistyka)
- Trading algorytmiczny

Zalety:

- ✅ Uczy się przez doświadczenie - nie wymaga oznaczonych przykładów

- ✅ Potrafi rozwiązywać bardzo złożone problemy sekwencyjne
- ✅ Może odkrywać nieoczekiwane strategie

Wady:

- ❌ Wymaga OGROMNEJ liczby prób (miliony iteracji)
- ❌ Bardzo zasobożerne obliczeniowo
- ❌ Trudne do debugowania

> ⚠️ **Uwaga**
>
> Reinforcement Learning to najtrudniejsze podejście - używane do najbardziej złożonych problemów!

Porównanie podejść do uczenia

Podejście	Dane	Typowe zastosowania	Trudność
Supervised	Oznaczone (X, y)	Klasyfikacja, regresja	⭐⭐
Unsupervised	Nieoznaczone (tylko X)	Klasteryzacja, anomalie	⭐⭐⭐
Semi-Supervised	Trochę oznaczonych + dużo nieoznaczonych	Rozpoznawanie obrazów	⭐⭐⭐
Reinforcement	Nagrody/kary za akcje	Gry, roboty, autonomiczne systemy	⭐⭐⭐⭐⭐

> 📝 **Notatka**
>
> W praktyce ~80% projektów to supervised learning, ~15% unsupervised, ~5% pozostałe podejścia.

---

4. 🌱 Aspekt środowiskowy

> 📚 **Definicja**
>
> **Green AI** to podejście do rozwoju sztucznej inteligencji i Data Science, które minimalizuje negatywny wpływ na środowisko poprzez optymalizację zużycia energii, zasobów obliczeniowych i danych.

Wprowadzenie - dlaczego to ważne?

Data Science i Machine Learning mają ogromny potencjał do rozwiązywania problemów środowiskowych, ale same mogą być energochłonne. Świadomy Data Scientist musi rozumieć ten trade-off i działać odpowiedzialnie.

Wpływ ML/AI na środowisko

Fakty o zużyciu energii:

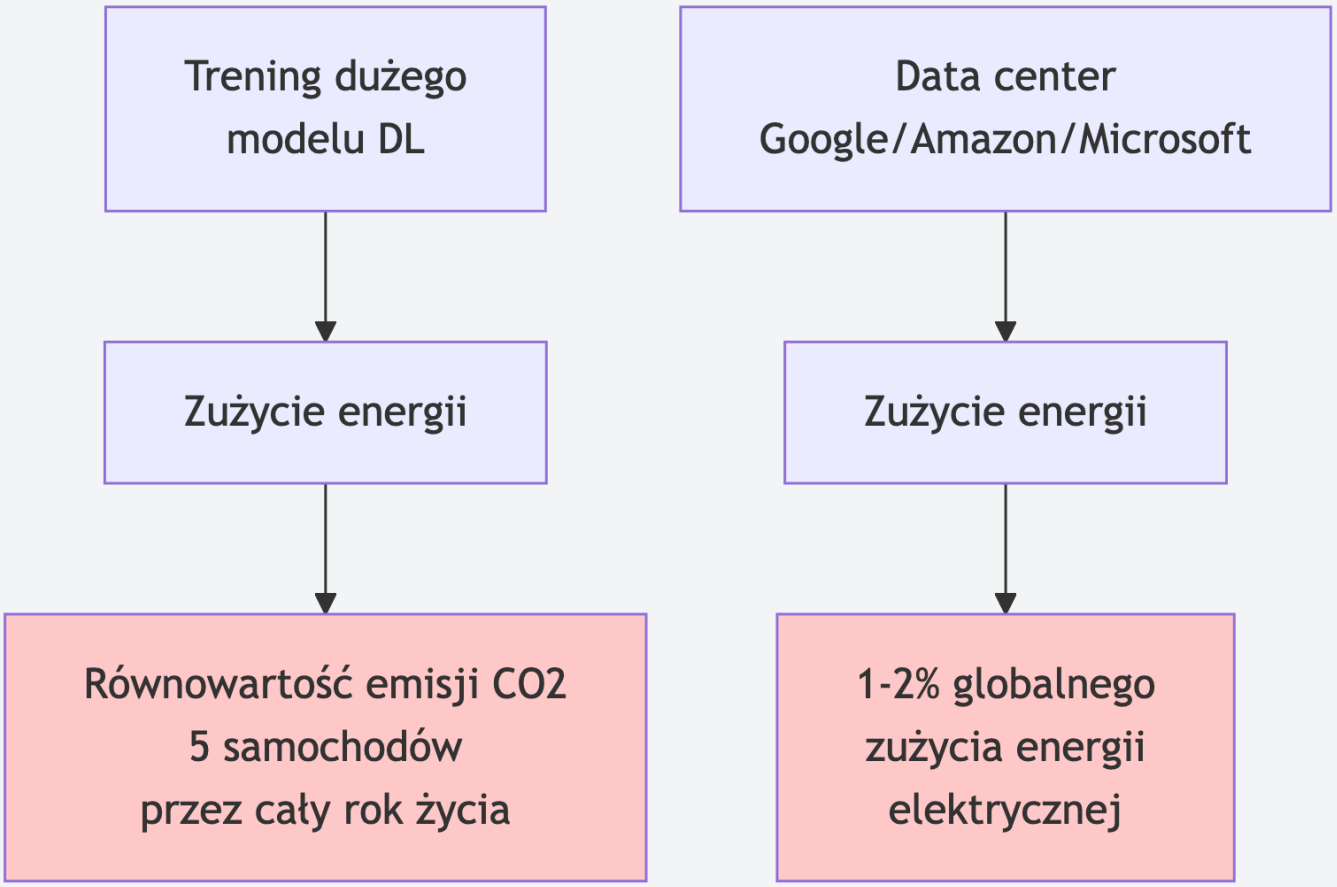


Diagram 17

Przykłady:

- Trening dużego modelu językowego (np. GPT-3): ~1,300 MWh energii
- To równowartość emisji CO2 jak 5 samochodów przez całe życie!
- Data centers zużywają ~1-2% globalnej energii elektrycznej

> ⚠ **Uwaga**
>
> Im większy model i więcej danych, tym większe zużycie energii i emisja CO2!

Porównanie efektywności energetycznej różnych podejść

1. Systemy regułowe (tradycyjne programowanie)

```
# Bardzo energooszczędne - prosta logika
def is_spam_rules(email):
    if 'viagra' in email or 'casino' in email:
        return True
    return False

# Zużycie: ~0.001 Wh na milion klasyfikacji
```

✅ **Najmniej energii**, ale ograniczona skuteczność

2. Klasyczne ML (np. regresja logistyczna, drzewa decyzyjne)

```
# Średnie zużycie energii
model = LogisticRegression()
model.fit(X_train, y_train)  # Trening: kilka sekund na CPU

# Zużycie treningu: ~0.1 Wh
# Zużycie predykcji: ~0.01 Wh na milion klasyfikacji
```

✅ **Dobry balans: skuteczność / energia**

3. Deep Learning (sieci neuronowe)

```
# Wysokie zużycie energii
neural_network = build_large_model()
neural_network.fit(X_train, y_train, epochs=100)  # Trening: godziny/dni na GPU

# Zużycie treningu: 10-1000 Wh (zależnie od rozmiaru)
# Zużycie predykcji: ~1 Wh na milion klasyfikacji
```

⚠️ **Najwyższa skuteczność, ale największe zużycie energii**

Trade-off: Dokładność vs Energia

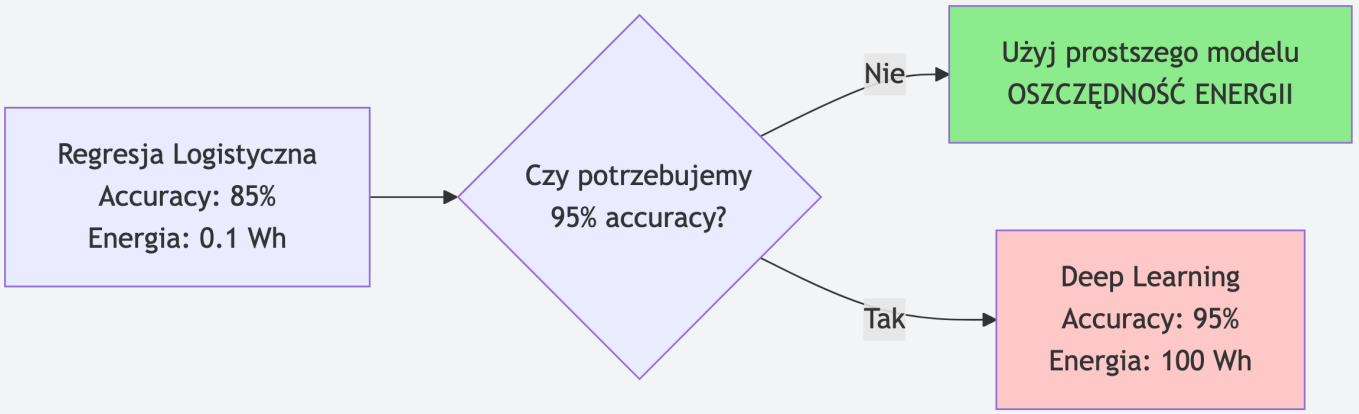


Diagram 18

> 💡 **Wskazówka**
>
> **Zasada:** Użyj najprostszego modelu, który osiąga wystarczającą dokładność. Nie strzelaj armatą do muchy!

Zasady Green Data Science

1. Próbkuj dane podczas testowania

```
# ❌ ZŁE: Używamy wszystkich 10 milionów rekordów do testowania
full_data = load_data(10_000_000)
test_model(full_data)  # Długo trwa, dużo energii

# ✅ DOBRE: Próbkujemy reprezentatywny subset
sample_data = load_data(10_000_000).sample(100_000)  # 1% danych
test_model(sample_data)  # Szybko, mało energii, wyniki prawie identyczne
```

2. Optymalizuj hyperparametry mądrze

```
# ❌ ZŁE: Grid Search - testujemy WSZYSTKIE kombinacje
param_grid = {
    'n_estimators': [100, 200, 300, 400, 500],  # 5 wartości
    'max_depth': [10, 20, 30, 40, 50],          # 5 wartości
    'learning_rate': [0.01, 0.05, 0.1, 0.2]     # 4 wartości
}
# Total: 5 * 5 * 4 = 100 trenowań modelu! 💸💸💸

# ✅ DOBRE: Bayesian Optimization - inteligentne przeszukiwanie
# Testuje tylko ~20 kombinacji, znajduje prawie optymalne rozwiązanie
bayesian_search(param_space, n_iterations=20)
```

3. Transfer Learning - wykorzystuj gotowe modele

```
# ❌ ZŁE: Trenuj model od zera
model = build_resnet50_from_scratch()
model.train(1_000_000_images, epochs=100)  # Dni treningu, tysiące Wh

# ✅ DOBRE: Użyj wytrenowanego modelu i dostosuj
pretrained_model = load_resnet50_pretrained()  # Model już nauczony!
pretrained_model.fine_tune(my_10000_images, epochs=5)  # Kilka godzin, kilkadziesiąt Wh
```

4. Monitoruj zasoby

```
# Monitoruj zużycie CPU/GPU/RAM podczas treningu
import psutil
import GPUtil

def monitor_resources():
    cpu_percent = psutil.cpu_percent()
    memory = psutil.virtual_memory().percent
    gpus = GPUtil.getGPUs()

    print(f"CPU: {cpu_percent}%")
    print(f"RAM: {memory}%")
    if gpus:
        print(f"GPU: {gpus[0].load * 100}%")

    # Jeśli zużycie > 90%, może warto zmniejszyć batch size?
```

5. Zamykaj niepotrzebne procesy

```
# ✅ Pamiętaj o zamykaniu instancji chmurowych po treningu!

# Po zakończeniu treningu
model.save('model.pkl')

# Wyłącz instancję GPU w chmurze (AWS/GCP)
shutdown_gpu_instance()  # Nie płacisz, nie marnujesz energii
```

Monitoring zasobów - praktyczne narzędzia

Przykład - śledzenie zużycia pamięci:

```
# Mierzymy ile pamięci zużywa operacja
import sys

data_small = list(range(1000))
data_large = list(range(1_000_000))

print(f"Małe dane: {sys.getsizeof(data_small) / 1024} KB")
# Wynik: ~8 KB

print(f"Duże dane: {sys.getsizeof(data_large) / 1024 / 1024} MB")
# Wynik: ~8 MB

# Wniosek: Jeśli nie potrzebujesz miliona rekordów, użyj mniejszego zestawu!
```

Efficiency Ratio: Dokładność / Energia

```
# Porównujemy modele według efficiency ratio

model_a = {
    'name': 'Logistic Regression',
    'accuracy': 0.85,
    'energy_wh': 0.1
}

model_b = {
```

```
    'name': 'Deep Neural Network',
    'accuracy': 0.88,
    'energy_wh': 100
}

# Efficiency Ratio = Accuracy / Energy
er_a = model_a['accuracy'] / model_a['energy_wh']  # 8.5
er_b = model_b['accuracy'] / model_b['energy_wh']  # 0.0088

print(f"Model A efficiency: {er_a}")
print(f"Model B efficiency: {er_b}")
print(f"Model A jest {er_a / er_b:.0f}x bardziej energooszczędny!")
# Wynik: Model A jest 966x bardziej energooszczędny!
```

> ✅ **Sukces**
>
> **Dobry Data Scientist** zawsze rozważa trade-off między dokładnością modelu a zużyciem zasobów!

Jak Data Science może pomagać środowisku?

Pozytywne zastosowania DS dla ekologii:

1. **Monitoring jakości powietrza**

- Przewidywanie poziomów zanieczyszczeń
- Identyfikacja źródeł emisji
- Optymalizacja tras transportu publicznego

1. **Optymalizacja zużycia energii**

- Smart buildings - dostosowanie ogrzewania/klimatyzacji
- Prognozowanie zapotrzebowania na energię
- Optymalizacja sieci energetycznych

1. **Smart Agriculture**

- Optymalne nawodnienie (oszczędność wody)
- Precyzyjne dawkowanie nawozów
- Przewidywanie plonów

1. **Zarządzanie odpadami**

- Optymalizacja tras śmieciarek
- Klasyfikacja odpadów do recyklingu
- Przewidywanie ilości odpadów

1. **Ochrona przyrody**

- Monitoring populacji zagrożonych gatunków
- Przewidywanie pożarów lasów
- Analiza zmian klimatycznych

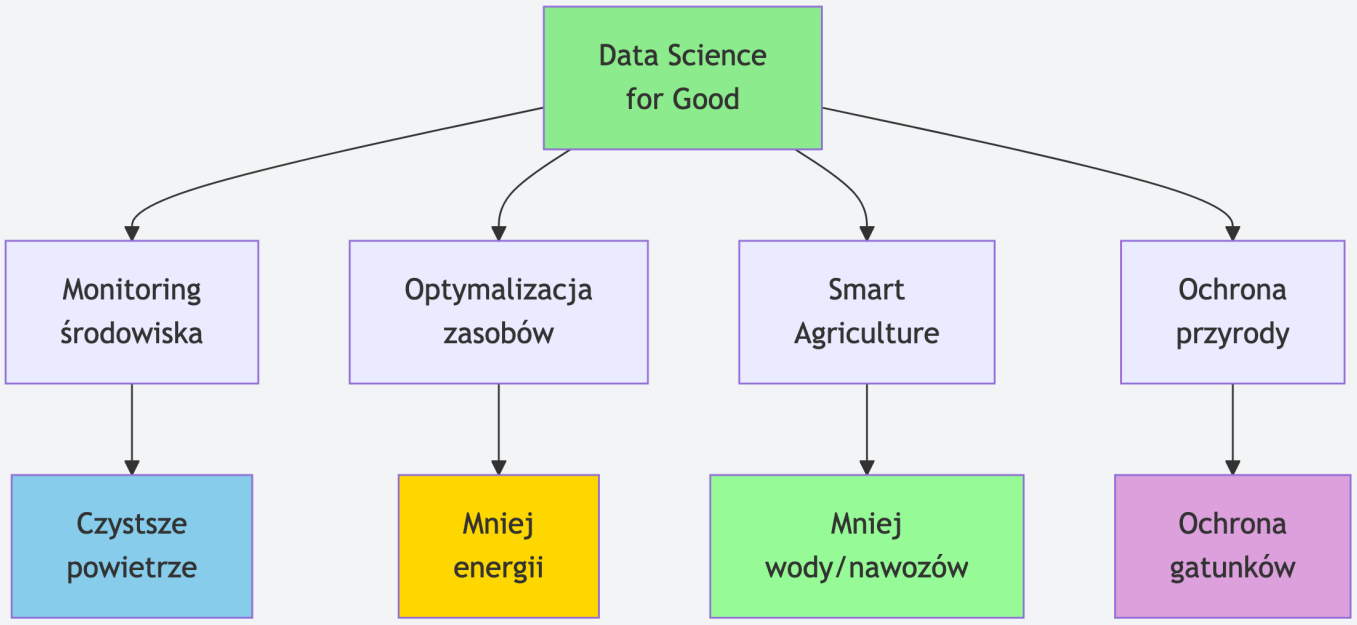


Diagram 19

Kluczowe zasady Green Data Science

1. **Użyj najprostszego modelu**, który daje wystarczającą dokładność
2. **Próbkuj dane** podczas eksperymentów
3. **Optymalizuj hyperparametry mądrze** (Bayesian zamiast Grid Search)
4. **Wykorzystuj transfer learning** zamiast trenować od zera
5. **Monitoruj zużycie zasobów** (CPU/GPU/RAM)

6. **Zamykaj niepotrzebne procesy** i instancje chmurowe
7. **Rozważaj trade-off**: czy 3% lepszej accuracy warte jest 100x więcej energii?

> 📝 Notatka
>
> **Pamiętaj:** Data Science może służyć środowisku, ale tylko jeśli sami będziemy działać odpowiedzialnie!

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

5. 🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Identyfikacja typu zadania ML**

Dla każdego z poniższych problemów określ, jaki to typ zadania ML:

- Przewidywanie ceny samochodu na podstawie przebiegu i roku produkcji
- Grupowanie artykułów w sklepie internetowym według podobieństwa

- Wykrywanie twarzy na zdjęciu z kamery
- Klasyfikacja emaili jako spam/nie-spam
- Generowanie opisów produktów na podstawie zdjęć

Wymagania:

- Podaj typ zadania dla każdego przykładu
- Uzasadnij, dlaczego to właśnie ten typ

(proste)

1. ✏️ **Zadanie 2 – Rodzaje uczenia**

Określ, jakie podejście do uczenia (supervised/unsupervised/semi-supervised/reinforcement) jest najbardziej odpowiednie dla:

- Segmentacja klientów sklepu (brak danych o grupach)
- Klasyfikacja zdjęć kotów vs psów (mamy 10,000 opisanych zdjęć)
- Nauczenie robota chodzenia
- Rozpoznawanie twarzy (mamy 1,000 opisanych + 100,000 nieopisanych zdjęć)

Wymagania:

- Uzasadnij wybór podejścia
- Wyjaśnij, dlaczego inne podejścia byłyby gorsze

(proste)

1. ✏️ **Zadanie 3 – Regresja vs Klasyfikacja**

Dla każdego problemu określ, czy to regresja czy klasyfikacja:

- Przewidywanie liczby sprzedanych produktów w następnym miesiącu
- Określenie, czy klient zrezygnuje z usługi (tak/nie)
- Szacowanie czasu dostawy paczki
- Klasyfikacja gatunku kwiatu (Iris setosa/versicolor/virginica)
- Przewidywanie ceny akcji za tydzień

Wymagania:

- Uzasadnij swój wybór

- Podaj przykład możliwego wyniku modelu

(proste)

1. ✏️ **Zadanie 4 – Analiza case study: spam filter**

Wyobraź sobie, że tworzysz system filtrowania spamu dla firmy email.

Pytania:

- Jakie dane potrzebujesz do treningu modelu?
- Jaki typ zadania ML to jest?
- Jakie podejście do uczenia zastosujesz?
- Jakie cechy (features) emaila mogą być użyteczne?
- Jak ocenisz, czy model działa dobrze?

(proste)

1. ✏️ **Zadanie 5 – Green AI: analiza zużycia energii**

Masz do wyboru dwa modele:

- Model A: accuracy 92%, czas treningu 10 minut na CPU
- Model B: accuracy 94%, czas treningu 12 godzin na GPU

Pytania:

- Który model wybierzesz i dlaczego?
- W jakich sytuacjach warto wybrać Model B mimo większego zużycia energii?
- Jakie czynniki poza accuracy i energią warto rozważyć?

(proste)

1. ✏️ **Zadanie 6 – Supervised vs Unsupervised**

Masz bazę 100,000 klientów sklepu. Chcesz ich podzielić na grupy w celu personalizacji ofert.

Scenariusz A: Masz dane o klientach (wiek, dochód, historia zakupów), ale NIE masz gotowych kategorii.
Scenariusz B: Masz dane o klientach I ręcznie sklasyfikowałeś 10,000 z nich do 5 kategorii.

Pytania:

- Jakie podejście (supervised/unsupervised) użyjesz w każdym scenariuszu?

- Jakie są zalety i wady każdego podejścia?
- Który scenariusz daje lepsze wyniki?

(proste)

1. ✏️ **Zadanie 7 – Transfer Learning**

Firma farmaceutyczna chce klasyfikować zdjęcia komórek (zdrowe/chore). Ma 2,000 opisanych zdjęć.

Opcja A: Trenuj model od zera na 2,000 zdjęciach Opcja B: Użyj gotowego modelu wytrenowanego na milionach zdjęć medycznych i dostosuj do swojego problemu

Pytania:

- Która opcja zużyje mniej energii?
- Która da lepsze wyniki?
- Dlaczego transfer learning to podejście "Green AI"?

(proste)

1. ✏️ **Zadanie 8 – Przykłady zastosowań DS dla ekologii**

Wymyśl 3 przykłady projektów Data Science, które mogą pozytywnie wpłynąć na środowisko.

Dla każdego projektu określ:

- Problem do rozwiązania
- Typ zadania ML (regresja/klasyfikacja/etc.)
- Jakie dane są potrzebne
- Jaki będzie pozytywny wpływ na środowisko

(proste)

✏️ Zadania średnie (9-12)

1. ✏️ **Zadanie 9 – Projektowanie systemu rekomendacji**

Zaprojektuj system rekomendacji filmów dla platformy streamingowej.

Wymagania:

- Określ, jaki typ zadania ML to jest (klasyfikacja/regresja/ranking/inne)
- Jakie dane potrzebujesz (o użytkownikach, filmach, interakcjach)

- Jakie podejście do uczenia zastosujesz i dlaczego
- Zaproponuj metrykę do oceny jakości rekomendacji
- Jak zoptymalizujesz system pod kątem Green AI?

Oczekiwany rezultat:

- Dokument z architekturą systemu
- Uzasadnienie decyzji technicznych
- Plan optymalizacji energetycznej

(średnie)

1. ✏️ **Zadanie 10 – Analiza trade-off: dokładność vs energia**

Przeprowadź analizę dla problemu klasyfikacji obrazów:

Model 1: Logistic Regression - 75% accuracy, 0.5 Wh Model 2: Random Forest - 85% accuracy, 5 Wh Model 3: ResNet50 - 92% accuracy, 500 Wh Model 4: EfficientNet - 90% accuracy, 50 Wh

Zadania:

- Oblicz efficiency ratio (accuracy/energia) dla każdego modelu
- Stwórz wykres (konceptualny) pokazujący trade-off
- W jakich przypadkach każdy model jest najlepszy?
- Który model wybierzesz dla aplikacji mobilnej? Dlaczego?
- Który model wybierzesz dla systemu bezpieczeństwa w elektrowni jądrowej? Dlaczego?

(średnie)

1. ✏️ **Zadanie 11 – Design case: wykrywanie fraudu**

Bank chce wykrywać oszukańcze transakcje kartą kredytową.

Dane:

- 10 milionów transakcji dziennie
- ~0.1% transakcji to fraud (1000 na milion)
- Cechy: kwota, lokalizacja, czas, typ sklepu, historia klienta

Zaprojektuj system:

- Jaki typ zadania ML?

- Supervised czy unsupervised? Uzasadnij
- Jakie są wyzwania (np. imbalanced data)?
- Jak ocenisz model (accuracy? precision? recall? inne?)
- Jak zapewnisz, że system jest energooszczędny (10M transakcji/dzień!)?

Oczekiwany rezultat:

- Dokument z architekturą rozwiązania
- Analiza wyzwań i sposobów ich rozwiązania

(średnie)

1. ✏️ **Zadanie 12 – Comparison: reguły vs ML**

Porównaj tradycyjne programowanie (reguły) vs Machine Learning dla problemu: "Automatyczna moderacja komentarzy na forum (filtrowanie obraźliwych treści)"

Wymagania:

- Zaprojektuj rozwiązanie oparte na regułach
- Zaprojektuj rozwiązanie oparte na ML
- Porównaj: dokładność, łatwość utrzymania, zużycie energii, koszt
- Która metoda jest lepsza i dlaczego?
- Czy można połączyć oba podejścia? Jak?

Oczekiwany rezultat:

- Tabela porównawcza
- Uzasadnienie wyboru
- Propozycja hybrydowego rozwiązania

(średnie)

🧠 Zadania wyzwanie (13-20)

1. 🧠 **Zadanie 13 – Design end-to-end: Smart City Traffic**

Miasto chce zoptymalizować sygnalizację świetlną, aby zmniejszyć korki i emisję CO2.

Zaprojektuj kompletny system Data Science:

- Analiza problemu: jakie dane potrzebujesz?
- Jakie typy zadań ML będą potrzebne? (może być kilka!)
- Jakie podejścia do uczenia?
- Jak zmierzysz sukces projektu?
- Jak zapewnisz, że system jest Green AI?
- Jakie są ryzyka i wyzwania etyczne?

Oczekiwany rezultat:

- Kompleksowy dokument projektowy
- Diagram przepływu danych
- Metryki biznesowe i techniczne
- Analiza wpływu środowiskowego

(challenge)

1. 🧠 **Zadanie 14 – Semi-Supervised Strategy**

Masz 100,000 zdjęć produktów w sklepie, ale tylko 1,000 jest opisanych (np. "buty", "koszula", "spodnie").

Zaprojektuj strategię semi-supervised learning:

- Jak wykorzystasz 1,000 opisanych zdjęć?
- Jak wykorzystasz 99,000 nieopisanych zdjęć?
- Zaproponuj konkretny algorytm/metodologię
- Jak porównasz wyniki z baseline (tylko 1,000 opisanych)?
- Oszacuj oszczędność energii vs trenowanie modelu na pełnych 100,000 opisanych zdjęć

Oczekiwany rezultat:

- Plan projektu semi-supervised learning
- Porównanie z alternatywami
- Analiza ROI (dokładność / koszt / energia)

(challenge)

1. 🧠 **Zadanie 15 – Reinforcement Learning Design**

Zaprojektuj agenta RL do zarządzania energią w inteligentnym budynku.

Cel: Minimalizacja zużycia energii przy zachowaniu komfortu mieszkańców.

Zdefiniuj:

- Agent: co kontroluje? (ogrzewanie, klimatyzacja, oświetlenie?)
- Environment: jakie są stany? (temperatura, pora dnia, obecność ludzi?)
- Actions: jakie akcje może wykonać agent?
- Rewards: jak nagradzać/karać agenta?
- Czy można to rozwiązać supervised learning? Dlaczego RL jest lepsze?

Oczekiwany rezultat:

- Formalny opis problemu RL
- Diagram agent-environment loop
- Funkcja nagrody (reward function)
- Porównanie z podejściem supervised

(challenge)

1. 🧠 **Zadanie 16 – Multi-Model System Design**

E-commerce chce systemu, który:

1. Klasyfikuje zdjęcia produktów (co to jest?)
2. Generuje opisy produktów
3. Rekomenduje produkty użytkownikom
4. Wykrywa fraudulentne recenzje

Zaprojektuj kompletny system ML:

- Jakie typy zadań ML dla każdego problemu?
- Jakie podejścia do uczenia?
- Jak te systemy będą współpracować?
- Które komponenty najbardziej energochłonne?
- Jak zoptymalizować całość pod kątem Green AI?
- Zaproponuj architekturę systemu (diagram)

Oczekiwany rezultat:

- Diagram architektury

- Opis każdego komponentu
- Analiza zużycia zasobów
- Plan optymalizacji

(challenge)

1. 🧠 **Zadanie 17 – Benchmark: Energy vs Accuracy**

Przeprowadź teoretyczny benchmark 5 różnych podejść do klasyfikacji tekstu:

1. Keyword matching (reguły)
2. Logistic Regression (TF-IDF)
3. Random Forest
4. LSTM (Recurrent Neural Network)
5. BERT (Transformer)

Dla każdego podejścia oszacuj (na podstawie research):

- Spodziewana accuracy
- Zużycie energii podczas treningu
- Zużycie energii podczas predykcji
- Czas treningu
- Czas predykcji
- Łatwość wdrożenia

Oczekiwany rezultat:

- Tabela porównawcza
- Wykres accuracy vs energia
- Rekomendacja: które podejście w jakim scenariuszu
- Analiza Green AI efficiency

(challenge)

1. 🧠 **Zadanie 18 – Ethical AI + Green AI**

Szpital chce stworzyć system ML do diagnostyki chorób z obrazów RTG.

Rozważ:

- Aspekty etyczne: co jeśli model się myli? Kto ponosi odpowiedzialność?
- Trade-off accuracy vs energia: czy 99% accuracy warte jest 100x więcej energii niż 97%?
- Fairness: czy model działa równie dobrze dla różnych grup demograficznych?
- Privacy: jak chronić dane pacjentów?
- Green AI: jak zminimalizować carbon footprint systemu?

Oczekiwany rezultat:

- Dokument analizujący wszystkie aspekty
- Propozycja rozwiązań dla każdego wyzwania
- Framework decyzyjny (kiedy trade-off accuracy/energia jest akceptowalny)

(challenge)

1. 🧠 **Zadanie 19 – Data Science for Climate Change**

Zaprojektuj projekt DS do walki ze zmianami klimatycznymi.

Wymagania:

- Wybierz konkretny problem (np. prognozowanie pożarów lasów, optymalizacja energii odnawialnej, monitoring wylesiania)
- Określ typ zadania ML i podejście do uczenia
- Jakie dane potrzebujesz i skąd je pozyskasz?
- Jak zmierzysz impact projektu? (np. ile ton CO_2 zaoszczędzono)
- Jak zapewnisz, że sam projekt jest Green AI?
- Oszacuj ROI: korzyści środowiskowe vs koszt (energia/pieniądze)

Oczekiwany rezultat:

- Kompleksowy plan projektu
- Analiza danych i metodologii
- Impact assessment framework
- Prezentacja dla stakeholderów

(challenge)

1. 🧠 **Zadanie 20 – Future of Green AI**

Napisz esej/raport (1000-1500 słów) na temat: "Jak będzie wyglądał Green AI w 2030 roku?"

Rozważ:

- Trendy w efektywności energetycznej modeli ML
- Nowe technologie (kwantowe komputery, neuromorphic chips?)
- Regulacje prawne dotyczące carbon footprint AI
- Trade-off między innowacją a ochroną środowiska
- Rola Data Scientists w zrównoważonym rozwoju AI
- Przykłady przełomowych projektów Green AI

Oczekiwany rezultat:

- Dokument 1000-1500 słów
- Bibliografia (minimum 5 źródeł)
- Wizja przyszłości oparta na research i trendach
- Rekomendacje dla obecnych Data Scientists

(challenge)

---

6. 📚 Podsumowanie

Czego nauczyliśmy się w tej lekcji?

W pierwszej lekcji kursu Data Science poznaliśmy fundamenty, na których zbudujesz swoją karierę:

1. Ekosystem AI/ML/DS:

- **AI (Sztuczna Inteligencja)** - najszersze pojęcie, systemy wykonujące zadania wymagające inteligencji
- **ML (Machine Learning)** - podzbiór AI, systemy uczące się z danych
- **DS (Data Science)** - interdyscyplinarna dziedzina łącząca statystykę, programowanie, ML i biznes

2. Typy zadań Machine Learning:

- **Regresja** - przewidywanie liczb (cena, temperatura, sprzedaż)
- **Klasyfikacja** - przypisanie do kategorii (spam/nie-spam, zdrowy/chory)
- **Klasteryzacja** - grupowanie podobnych obiektów (segmentacja klientów)
- **Rangowanie** - uporządkowanie według trafności (wyszukiwarka, rekomendacje)
- **Detekcja** - wykrywanie obiektów i ich lokalizacja (twarze, samochody)

- **Segmentacja** - podział obrazu na regiony (medycyna, autonomiczne pojazdy)
- **Generowanie** - tworzenie nowych danych (ChatGPT, DALL-E)

3. Podejścia do uczenia modeli:

- **Supervised Learning** - uczenie z danych oznaczonych (najczęstsze, ~80% projektów)
- **Unsupervised Learning** - uczenie z danych nieoznaczonych (odkrywanie wzorców)
- **Semi-Supervised Learning** - kombinacja obu (mało oznaczonych + dużo nieoznaczonych)
- **Reinforcement Learning** - uczenie przez nagrody/kary (gry, roboty)

4. Green AI i aspekty środowiskowe:

- Trenowanie dużych modeli zużywa ogromne ilości energii
- Trade-off: **dokładność vs energia** - nie zawsze najlepszy model jest potrzebny
- Zasady Green Data Science: próbkowanie, optymalizacja, transfer learning, monitoring
- Data Science może **pomagać środowisku**: optymalizacja energii, monitoring zanieczyszczeń, smart agriculture

Kluczowe wnioski

> ✅ **Sukces**
>
> **Data Science to nie tylko technologia** - to narzędzie do rozwiązywania realnych problemów ludzi, biznesów i planety.

> 💡 **Wskazówka**
>
> **Dobry Data Scientist** zawsze myśli o trade-off między skutecznością modelu a zużyciem zasobów. Nie strzelaj armatą do muchy!

> 📝 **Notatka**
>
> **Rozpoznanie typu zadania** to pierwszy krok w każdym projekcie ML - determinuje wybór algorytmu, danych i metodologii.

Jak to się łączy z dalszymi lekcjami?

Ta lekcja to fundament całego kursu:

- **Następne lekcje** nauczą Cię programowania w Python - języka Data Science
- **Później** poznasz narzędzia do pracy z danymi (NumPy, Pandas)
- **Potem** nauczysz się statystyki i wizualizacji
- **Wreszcie** zaimplementujesz algorytmy ML, o których dziś czytałeś teoretycznie

Wszystkie koncepcje z tej lekcji będą powracać w praktyce przez cały kurs!

Co dalej?

W następnej lekcji:

- Skonfigurujemy środowisko pracy (Google Colab / Jupyter Notebook)
- Poznasz podstawy pracy z notatnikami
- Zainstalujemy niezbędne narzędzia
- Rozpoczniemy naukę Python!

> *"Data is the new oil, but Data Science is the combustion engine." ~ (Dane to nowa ropa, ale Data Science to silnik spalinowy.)*
>
> — Cytat

Refleksja nad aspektem środowiskowym

Pamiętaj, że jako przyszły Data Scientist masz **odpowiedzialność**:

- Technologia, którą tworzysz, może wspierać lepsze decyzje proekologiczne
- Ale same modele ML mogą być energochłonne
- Twoja rola to znajdowanie **balansu** między innowacją a ochroną środowiska
- **Green AI to nie opcja, to konieczność** w świecie walki ze zmianami klimatycznymi

---

KURS DATA SCIENTIST ONLINE + AI

Chcesz zostać Data Scientist?

Ten materiał to fragment kursu Data Scientist Online + AI. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

7. 📖 Dodatkowe Zasoby

Rekomendowane artykuły i książki

Książki (początkujący):

- "The Hundred-Page Machine Learning Book" - Andriy Burkov
- "Data Science for Business" - Foster Provost, Tom Fawcett
- "Hands-On Machine Learning" - Aurélien Géron

Artykuły i blogi:

- Google AI Blog - aktualności z AI/ML
- Towards Data Science - praktyczne tutoriale
- Distill.pub - wizualne wyjaśnienia ML

Green AI:

- "Energy and Policy Considerations for Deep Learning in NLP" - Emma Strubell et al.
- Green AI - Paper o efektywności energetycznej AI
- ML CO2 Impact Calculator - oszacuj carbon footprint swojego modelu

Interaktywne narzędzia

- TensorFlow Playground - wizualizacja sieci neuronowych
- Seeing Theory - wizualne wprowadzenie do statystyki
- R2D3 - wizualne wprowadzenie do ML

Datasety do praktyki

- Kaggle Datasets - tysiące datasetów do eksperymentów
- UCI ML Repository - klasyczne datasety ML (Iris, Wine, etc.)
- Google Dataset Search - wyszukiwarka datasetów

Społeczności

- Kaggle - konkursy DS, notebooki, nauka
- r/MachineLearning - aktualności ML
- r/datascience - kariera, projekty, pytania

🎉 **Gratulacje! Ukończyłeś pierwszą lekcję kursu Data Science!**

W następnej lekcji **skonfigurujemy środowisko pracy** i **rozpoczniemy programowanie w Python**. Do zobaczenia! 🚀

ROZPOCZNIJ KARIERĘ W DATA SCIENTIST

Gotowy na następny krok?

Ten materiał to tylko początek. Dołącz do kursu Data Scientist Online + AI i zdobądź praktyczne umiejętności pod okiem ekspertów.

- 5,5 miesiąca intensywnej nauki
- Zajęcia na żywo 2x w tygodniu
- Certyfikat + pomoc w znalezieniu pracy

Zobacz szczegóły kursu →

Zadzwoń do nas

info@learnit.com.pl +48 573-580-990 learnit.com.pl